Algoritmo Topological-Sort, Single-Source Shortest Path

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 17, 2024
Studiato	Done

Sia $G = (V, E)$ un grafo. Diamo la definizione di **ordinamento topologico** di G.

> *Prende il nome di **ordinamento topologico** di un grafo l'ordinamento lineare dei suoi vertici tale che se* $(u, v) \in E \implies u < v$.

L'ordinamento è lineare perché possiamo disporre tali vertici in una linea.

L'algoritmo **topological-sort** prende in input un grafo orientato aciclico e restituisce in output un ordinamento topologico di G. Tale algoritmo è definito **solo** su grafi orientati aciclici.

Da ora in poi ci riferiremo ad un grafo orientato aciclico con l'acronimo **DAG**, "direct aciclic graph".

Algoritmo Topological-sort

Quando si esegue una DFS si fa una ricerca in profondità. Quando scopriamo un vertice, lo coloriamo di grigio e teniamo traccia del tempo in cui scopriamo il vertice. Analogamente, quando abbiamo finito di esplorare tutti gli archi uscenti, coloriamo il vertice di nero e teniamo conto del momento in cui è finita l'esplorazione del vertice e di tutti gli archi uscenti.

Allora i passi sono seguenti:

1. invochiamo la DFS su G per calcolare il finish-time $v.t$ per ogni $v \in V$;
2. al termine dell'esplorazione di ogni vertice, lo inseriamo in una lista collegata L;
3. l'algoritmo restituisce la lista L.

Il tempo computazionale del topological sort è $\Theta(|V| + |E|)$, dato che la DFS impiega un tempo $\Theta(|E| + |V|)$ e l'inserimento di un vertice nella lista L (si intende inserimento in

testa) impiega $O(1)$.

Per dimostrare che topological-sort restituisce un ordinamento topologico, dimostriamo il seguente lemma.

> *Sia $G = (V, E)$ un grafo. G è un DAG $\iff$ DFS non genera archi all'indietro.*

Dimostriamo $\implies$). Supponiamo che una DFS su G produca un arco all'indietro (u, v). Se (u, v) è un arco all'indietro, allora v è un predecessore di u nella foresta G_π. Quindi G contiene un ciclo, e questo è assurdo perché stiamo supponendo che G sia un DAG.

Dimostriamo $\impliedby$). Supponiamo che G contenga un ciclo γ e facciamo vedere che in tal caso la DFS produce archi all'indietro. Sia v il primo vertice in γ ad essere scoperto dalla DFS. Al momento $v.d$ (discovery di v), i vertici di γ formano un cammino bianco da v al predecessore di v in γ; chiamiamo tale predecessore u. Per il **white-path-Theorem**, se abbiamo un cammino di vertici bianchi, questa ricerca va in questo cammino bianco. Di conseguenza per questo teorema u è discendente di v nella foresta G_π. Ma in questo modo l'arco (u, v) è un arco all'indietro, e questo è un assurdo in quanto nega l'ipotesi dell'implicazione.

Usiamo questo lemma per dimostrare che il topological-sort restituisce un corretto ordinamento topologico.

> *Sia $G = (V, E)$ un DAG. Allora $\mathrm{Topological\text{-}Sort(G)}$ produce un ordinamento topologico di G.*

Per dimostrare la correttezza di Topological-Sort è sufficiente mostrare che se $(u, v) \in E$ allora $u < v$. Questo è vero **se e solo se** $v.f < u.f$. Osserviamo che quando DFS esplora u, il vertice v non può essere grigio, altrimenti esisterebbe un arco da u a v che sarebbe un arco all'indietro e quindi G non sarebbe un DAG. Allora v può essere solo bianco o nero:

- se fosse nero allora v è stato già esplorato a $v.f$ è stato già determinato, quindi $v.f < u.f$;
- se fosse bianco, allora v diventa un discendente di u e quindi $v.f < u.f$.

Quindi il teorema è dimostrato.

Problema Single-Source Shortest Paths

In input sono dati un grafo $G = (V, E)$, un vertice $s \in V$ chiamato sorgente e una funzione di peso $w : E \to \mathbb{R}$. L'obiettivo del problema è trovare un cammino minimo da s a v per qualunque vertice $v \in V$, ovvero $p =< u_0 = s, u_1, ..., u_n = v >$ tale che:

- $(u_{i-1}, u_i) \in E \quad \forall i \in \{1, ..., n\}$;
- $w(p) = \sum_{i=1}^{n} w(u_{i-1}, u_i)$ minimo.

Definiamo la funzione:

$$\delta(s, v) = \begin{cases} \min\limits_{s \to^p v} w(p) = \min\limits_{u_0 = s, u_n = v} \sum\limits_{i=1}^{n} w(u_{i-1}, u_i) & \text{se esiste un cammino da s a v} \\ \infty & \text{se non esiste un cammino da } s \text{ a } v \end{cases}$$

Notiamo che ci possono essere pesi negativi:

- se G non contiene cicli di peso negativo, allora $\delta(s, v)$ è ben definito per ogni v;
- se G contiene un ciclo di peso negativo raggiungibile da s, allora $\delta(s, v)$ non è ben definito, infatti possiamo sempre trovare un cammino di peso inferiore ripetendo il ciclo. Pertanto, in tal caso, definiamo $\delta(s, v) = -\infty$.

Un algoritmo che risolve tale problema è l'algoritmo di **Bellman Ford**.

Algoritmo di Bellman-Ford

Tale algoritmo risolve il problema Single-Source Shortest Paths nel caso generale in cui i pesi degli archi possono essere negativi.

Dato un grafo $G = (V, E)$ orientato, con $s \in V$ vertice sorgente e $w : E \to \mathbb{R}$ funzione di peso, l'algoritmo Bellman-Ford restituisce un valore booleano:

- `false` significa che esiste un ciclo di peso negativo raggiungibile dalla sorgente;
- `true` significa che $\delta(s, v)$ è ben definita e in tal caso è possibile ricostruire il cammino da s a v per ogni $v \in V$.

```
Bellman-Ford(G, s, w):
   Initialize-Single-Source(G, s)
   for i = 1 to |V| - 1 do
      for (u,v) in E do
         relax(u, v, w)
   for each (u,v) in E do
      if v.d > u.d + w(u,v) then    // significa che abbiamo un ciclo negativo
         return false
```

```
    return true

Initialize-Single-Source(G, s):
    for each v in V do
        v.d = +inf
        v.pi = null
    s.d = 0

relax(u, v, w):
    if v.d > u.d + w(u,v) then
        v.d = u.d + w(u,v)
        v.pi = u
```

La procedura `Initialize-Single-Source` imposta la stima della distanza di s a 0, e ogni altra stima della distanza e ogni altro predecessore a `null`. In d poniamo la stima attuale del costo del percorso da s a v. Allora se $v.d > u.d + w(u, v)$ ne segue che conviene passare per u. La procedura `relax` di un arco (u, v) consiste nel verificare se, passando per u, è possibile migliorare il cammino minimo per v precedentemente trovato; in tal caso, aggiorniamo $v.d$ e $v.\pi$. Prende il nome di "relax" perché se $v.d \leq u.d + w(u, v)$ non occorre esercitare "pressione" affinché il vincolo sia rispettato, quindi il vincolo è "rilassato".

Se il grafo è un rappresentato con liste di adiacenza, la procedura impiega $O(|V|^2 + |V| \cdot |E|)$-Time. Se $|E| = \Omega(|V|)$, allora la procedura impiega $O(|V| \cdot |E|)$-Time. Con una matrice di adiacenza, richiede $O(|V|^3)$.

Si tratta di un O perché spesso in realtà sono sufficienti meno di $|V| - 1$ iterazioni del ciclo.